Conversation with Claude: Material Finder Project

CISC106 Final Project - Design Phase (LLM Conversation)

Transcript of the chat text. Tool actions (file creation, code tests) are not shown; the final version of the code that was created is included in the appendix.

User

[Attached files: a screenshot of the "Site Functionality Requirements" list, and the PDF "CISC106 Final Project 1 (Design)"]

Hey Claude I need help with a computer science project. I am allowed to use AI so don't worry. I need to create a website with python code with the drafter package. I want you to make sure all of the functions, routes, and pages you implement have appropriate doc strings and all of the variables and function have appropriate names. The website I need you to design is something I came up with so here is what I need from you. make sure to follow the restraints in the picture. I want to website to be a material finder. Basically the user would input 5 constraints and the program should give you a material from a set of materials defined in the website. The returned material should be the best fit for their constraints. If multiple materials work than it should return both saying either could work and if none of them work it should return cannot find suitable material. All of the specifications for the project and each part of the project are in the pdf document.

Claude

[Created the file material_finder.py]

I wrote `material_finder.py`, a Drafter site that follows your design and the requirements in your screenshot. I tested it by calling every route, and the server starts without errors. I haven't clicked through it in a browser. To run it, install Drafter with `pip install drafter` and run the file.

How it works

- The user enters five constraints: max weight, max budget, volume, stress (psi), and a location chosen from a dropdown.
- Each material's weight is its density times the volume, and its cost is that weight times its cost per pound.
- A material has to pass all five checks: weight, cost, strength, water resistance (for wet locations), and temperature range.
- If one material passes, it is the best material. If several pass, all are listed cheapest first with a note that any could work. If none pass, the page says "Cannot find suitable material."

Requirements checklist

- **Routes (13):** `index`, `instructions_page`, `constraint_page`, `find_material`, `materials_page`, `history_page`, `change_weight`, `change_cost`, `change_volume`, `change_stress`, `change_location`, `new_search`.
- **Pages (6):** home, instructions, constraint input, results, full material list, and past searches. The last one is an extra beyond your five.

- **State fields:** it has 8, including `materials: list[Material]`, `matches: list[Material]` and `history: list[SearchRecord]`. The weight, cost, volume, stress, location, `matches` and `history` fields are all changed by routes.
- **Second dataclass:** `Material`, with 20 instances. There are also `ProjectSite` and `SearchRecord`.
- **Input fields:** 4 TextBoxes and 1 SelectBox.
- **If statements:** five in `find_failed_constraints`, one per constraint, plus validation checks.
- **Loop:** `search_materials` loops over `state.materials` to test each one.
- **Globals:** only constants (`SITE_OPTIONS`, `WET_WEATHER`, `DRY_WEATHER`, `NO_MATCH_MESSAGE`), and nothing global is modified.
- **Docstrings and names:** every function has a docstring.

Changes from your written design

- Your PDF's `Location` dataclass is named `ProjectSite` in the code, because Drafter already has its own `Location` class and the names would clash.
- `cost_per_pound` is a float instead of a string, so the program can do math with it.
- Strength is one rough psi number per material. The material figures are approximate, so you may want to say that in your write-up.

Advanced technique: I used `list.sort(key=...)` to sort matches by cost. It is documented in the code, and you can swap it for a simple loop if you'd rather avoid it.

Still to do for the assignment

- Redraw your mockup so it shows the extra pages and routes.
- Update the PDF tables, including the fields I changed.
- Paste this conversation into the "LLM Conversation" section, since the instructions say you can get a zero if you leave it out.

I'd also suggest reading through the code until you can explain each part yourself.

User

I don't understand the return statement for the function on line 301 can you do it a different way that still works

Claude

[Edited material_finder.py]

Line 301 is `describe_constraints`. The confusing part is the f-string formatting codes like `:,.2f`, plus the adjacent strings inside parentheses that Python glues together. I'll replace it with plain string joining using `str()` and `round()`.

I rewrote `describe_constraints` in `material_finder.py`, and it still works. I ran a search and the summary came out correctly.

Each piece of the summary is now built on its own line, and the pieces are joined with +:

- `str(...)` turns a number into text, because you can't join a number to text with + directly.

- `round(..., 2)` keeps at most 2 decimal places.
- The five pieces are joined with `", "` between them, and the finished text is returned.

The history page will now show `$300.0` instead of `$300.00`, and it no longer puts commas in big numbers (`5000` instead of `5,000`). That is only a cosmetic difference in the history table.

Other functions, such as `build_results_page` and `materials_page`, still use f-strings with the same kind of formatting codes (`:,.2f`). If any of those are confusing too, tell me which line and I'll simplify them the same way.

User

I also don't understand the way you wrote the constraint_rows on line 380 can you change that

Claude

[Edited material_finder.py]

I rewrote `constraint_rows` in `build_results_page` in `material_finder.py`. I tested it, and the results page builds with the same table as before.

The old version put the formatting and the table inside one big list. The new version has two steps:

- **Make the text for each value.** Each constraint becomes a plain string using `str()`, `round()` and `+`, the same approach as `describe_constraints`. The location text is built up from its parts, such as `"Temperate Forest (50 F, Wet)"`.
- **Build the table one row at a time.** Start with an empty list and `.append()` each row. Each row is a small list with a label and its value. The first row holds the column headings.

Numbers on the results page now show as `500.0 lbs` instead of `500.00 lbs`, and `5000 psi` instead of `5,000 psi`.

Two other parts of the same function are written in the older style:

- `match_rows` still uses f-strings with `:,.2f`.
- `*outcome_content` uses the star to unpack a list into the page.

Tell me if you want either one changed too.

User

I don't understand every time you used f"{} with something inside the brackets. Can you change everytime you did this to an easier way like you have been doing

Claude

[Edited material_finder.py]

I'm still swapping out the f-strings, and the last ones are on the materials page, so I'm fixing those now.

I've removed every f-string from `material_finder.py`. A search for `f"` in the file finds nothing, and I tested the pages again with several matches, no matches, and the catalog page. They all work.

There were 11 places to change. All of them now use the same approach as the earlier edits: `str()` to turn a number into text, `round(number, 2)` to keep two decimal places, and `+` to join pieces together.

- **Results page:** The "only material" and "N materials meet all your constraints" messages are built with +. Each row of the matching-materials table gets its weight, cost and strength text from separate variables before it's added to the table.
- **Materials page:** Each row now builds its density, cost, strength and temperature range text in separate variables. The "any of these 20 materials" line is joined with +.

The only visible change is that numbers lose their commas and trailing zeros. For example, `$0.50` now shows as `$0.5`, `36,000` as `36000`, and `500.00 lbs` as `500.0 lbs`. The logic is the same.

If you'd like the `*outcome_content` line in the results page changed too, I can do that.

User

is there a way you could turn this conversation into a pdf

Claude

[Created the PDF Material_Finder_LLM_Conversation.pdf]

I made the PDF, `Material_Finder_LLM_Conversation.pdf`. It has your messages and my replies, with each speaker labeled, plus an appendix with the final version of `material_finder.py`. The behind-the-scenes actions, like running tests and creating files, are not shown; short bracketed notes mark where they happened.

The instructions say to paste the conversation into the "LLM Conversation" section of your design document. A PDF can't be pasted directly, so you can copy the text from this PDF into that section, or ask your instructor whether you can attach it.

User

i dont understand when you used the sort(key=...) on line 275 and beyond can you change it to something easier

Claude

[Edited material_finder.py]

I replaced `sort(key=...)` in `material_finder.py` with a simple loop in a new helper function, `sort_materials_by_cost`. I tested it, and the results are still listed cheapest first.

The new function repeats one simple step until the list is empty:

- Start by assuming the first material in the list is the cheapest.
- Loop through the rest, and if one costs less, make it the new cheapest.
- Move the cheapest material to the end of a new sorted list, and remove it from the old list.
- Repeat until no materials are left.

It only uses a `while` loop, a `for` loop, an `if` statement, `.append()` and `.remove()`. I also deleted the note about `sort(key=...)` from the top of the file, since the site no longer uses any advanced technique.

I also updated this PDF so the appendix shows the new code.

Appendix: Final Code (material_finder.py)

This is the final version of the file after all of the edits above.

```
"""
Material Finder
===============

A Drafter website that helps a user pick a construction / engineering material.

The user enters five constraints for their object:
    1. Maximum weight (lbs)
    2. Maximum budget (dollars)
    3. Volume of material needed (cubic feet)
    4. Stress the object must withstand (psi)
    5. Project location (which sets temperature range and wet/dry weather)

The site checks every material in its catalog against all five constraints.
  * If one material works, it is returned.
  * If several materials work, all of them are listed (cheapest first) and the
    site says that any of them could work.
  * If none work, the site says it cannot find a suitable material.

Simplifications (this is a school project, not an engineering tool):
  * "Strength" is a single rough number in psi (pounds per square inch) for each
    material, standing in for the stress at which it bends or breaks.
  * Material weight = density * volume.
  * Material cost   = weight * cost per pound.

The site uses no advanced coding techniques beyond what was covered in class.
"""

from dataclasses import dataclass
from drafter import *

# ------------------------------------------------------------------------------
# Constants (global constants are allowed; they are never changed)
# ------------------------------------------------------------------------------

WET_WEATHER = "Wet"
DRY_WEATHER = "Dry"
NO_MATCH_MESSAGE = "Cannot find suitable material."


# ------------------------------------------------------------------------------
# Dataclasses
# ------------------------------------------------------------------------------

@dataclass
class ProjectSite:
    """A place where the object will be used.

    Attributes:
        name: Display name of the site, such as "Hot Desert".
        average_temperature: Typical temperature at the site in degrees Fahrenheit.
        weather: Either WET_WEATHER or DRY_WEATHER.
    """
    name: str
    average_temperature: int
    weather: str


@dataclass
class Material:
    """A material the website can recommend.

    Attributes:
        name: Name of the material.
        density: Density in pounds per cubic foot.
        cost_per_pound: Price in dollars per pound.
        is_water_resistant: True if the material survives constant moisture.
```

```
        strength: Approximate stress in psi the material can take before it
            bends or breaks.
        min_temperature: Lowest safe temperature in degrees Fahrenheit.
        max_temperature: Highest safe temperature in degrees Fahrenheit.
    """
    name: str
    density: float
    cost_per_pound: float
    is_water_resistant: bool
    strength: int
    min_temperature: int
    max_temperature: int


@dataclass
class SearchRecord:
    """A short record of one past search, shown on the history page.

    Attributes:
        constraints_summary: One-line description of the constraints used.
        outcome: Names of the matching materials, or the no-match message.
    """
    constraints_summary: str
    outcome: str


@dataclass
class State:
    """Everything the website remembers between pages.

    Attributes:
        materials: The full catalog of materials the site can recommend.
        max_weight: Heaviest the finished object may be, in pounds.
        max_cost: Most the user can spend on material, in dollars.
        volume: Volume of material needed, in cubic feet.
        required_strength: Stress the object must withstand, in psi.
        site: The project site (temperature and weather) chosen by the user.
        matches: Materials that satisfied every constraint in the latest search.
        history: Records of every search made during this visit.
    """
    materials: list[Material]
    max_weight: float
    max_cost: float
    volume: float
    required_strength: int
    site: ProjectSite
    matches: list[Material]
    history: list[SearchRecord]


# -----------------------------------------------------------------------------
# Constant data: site choices
# -----------------------------------------------------------------------------

SITE_OPTIONS = [
    ProjectSite("Arctic Tundra", -20, DRY_WEATHER),
    ProjectSite("Alpine Mountains", 25, WET_WEATHER),
    ProjectSite("Temperate Forest", 50, WET_WEATHER),
    ProjectSite("Temperate Plains", 55, DRY_WEATHER),
    ProjectSite("Coastal Salt Air", 65, WET_WEATHER),
    ProjectSite("Tropical Rainforest", 85, WET_WEATHER),
    ProjectSite("Hot Desert", 105, DRY_WEATHER),
]


# -----------------------------------------------------------------------------
# Helper functions (not routes)
# -----------------------------------------------------------------------------

def build_material_catalog() -> list[Material]:
    """Create the list of 20 materials the website can recommend.
```

```
    Values are rounded, typical figures meant for a class project.

    Returns:
        A new list containing 20 Material instances.
    """
    return [
        Material("Structural Steel", 490.0, 0.50, False, 36000, -20, 800),
        Material("Stainless Steel", 500.0, 2.00, True, 31000, -300, 1500),
        Material("Aluminum 6061", 169.0, 1.50, True, 40000, -320, 400),
        Material("Titanium Alloy", 277.0, 25.00, True, 120000, -320, 800),
        Material("Copper", 559.0, 4.50, True, 10000, -320, 400),
        Material("Brass", 530.0, 3.50, True, 25000, -320, 400),
        Material("Cast Iron", 450.0, 0.40, False, 30000, -40, 750),
        Material("Red Oak Wood", 44.0, 1.00, False, 14000, -40, 300),
        Material("Douglas Fir Wood", 34.0, 0.60, False, 12000, -40, 300),
        Material("Pressure-Treated Pine", 36.0, 0.40, True, 8000, -40, 250),
        Material("Concrete", 150.0, 0.05, True, 4000, -50, 600),
        Material("Reinforced Concrete", 155.0, 0.10, True, 6000, -50, 600),
        Material("Granite", 165.0, 0.30, True, 15000, -50, 1000),
        Material("HDPE Plastic", 60.0, 1.00, True, 4000, -148, 176),
        Material("PVC Plastic", 87.0, 0.80, True, 7500, 14, 140),
        Material("Polycarbonate", 75.0, 2.50, True, 9000, -40, 250),
        Material("Acrylic", 74.0, 2.00, True, 10000, -40, 170),
        Material("Nylon", 71.0, 2.50, True, 11000, -40, 250),
        Material("Carbon Fiber Composite", 98.0, 30.00, True, 100000, -60, 250),
        Material("Fiberglass", 112.0, 3.00, True, 30000, -60, 300),
    ]


def is_positive_number(text: str) -> bool:
    """Check whether text is a positive number such as "12" or "3.5".

    Args:
        text: The raw text a user typed into a TextBox.

    Returns:
        True if the text is a number greater than zero, otherwise False.
    """
    cleaned_text = text.strip()
    # Remove at most one decimal point so "3.5" counts as a number but "3.5.1" does not.
    digits_only = cleaned_text.replace(".", "", 1)
    if not digits_only.isdigit():
        return False
    return float(cleaned_text) > 0


def find_site_by_name(site_name: str) -> ProjectSite:
    """Look up a ProjectSite from the list of site options by its name.

    Args:
        site_name: Name of a site, taken from the location SelectBox.

    Returns:
        The matching ProjectSite, or the first option if no name matches.
    """
    for site in SITE_OPTIONS:
        if site.name == site_name:
            return site
    return SITE_OPTIONS[0]


def calculate_weight(material: Material, volume: float) -> float:
    """Calculate how much a volume of a material weighs.

    Args:
        material: The material being used.
        volume: Volume of material in cubic feet.

    Returns:
        Weight in pounds.
    """
    return material.density * volume
```

```
def calculate_cost(material: Material, volume: float) -> float:
    """Calculate how much a volume of a material costs.

    Args:
        material: The material being used.
        volume: Volume of material in cubic feet.

    Returns:
        Total cost in dollars.
    """
    return calculate_weight(material, volume) * material.cost_per_pound


def find_failed_constraints(material: Material, state: State) -> list[str]:
    """Check one material against all five of the user's constraints.

    Args:
        material: The material to test.
        state: The current State holding the user's constraints.

    Returns:
        A list of short messages, one for each constraint the material fails.
        An empty list means the material satisfies every constraint.
    """
    failed_constraints = []

    # Constraint 1: weight
    if calculate_weight(material, state.volume) > state.max_weight:
        failed_constraints.append("too heavy")

    # Constraint 2: cost
    if calculate_cost(material, state.volume) > state.max_cost:
        failed_constraints.append("too expensive")

    # Constraint 3: stress
    if material.strength < state.required_strength:
        failed_constraints.append("not strong enough")

    # Constraint 4: weather (wet sites need a water resistant material)
    if state.site.weather == WET_WEATHER and not material.is_water_resistant:
        failed_constraints.append("not water resistant")

    # Constraint 5: temperature (site must be inside the material's safe range)
    if state.site.average_temperature < material.min_temperature:
        failed_constraints.append("too cold for this material")
    elif state.site.average_temperature > material.max_temperature:
        failed_constraints.append("too hot for this material")

    return failed_constraints


def sort_materials_by_cost(materials: list[Material],
                           volume: float) -> list[Material]:
    """Put materials in order from cheapest to most expensive.

    Works by repeating one simple step: find the cheapest material that is
    still left in the list, move it to the end of the sorted list, and repeat
    until no materials are left.

    Args:
        materials: The materials to sort. This list is emptied by the function.
        volume: Volume of material in cubic feet, used to calculate each cost.

    Returns:
        A new list with the same materials, cheapest first.
    """
    sorted_materials = []
    while len(materials) > 0:
        cheapest_material = materials[0]
        for material in materials:
```

```
            if (calculate_cost(material, volume)
                    < calculate_cost(cheapest_material, volume)):
                cheapest_material = material
        sorted_materials.append(cheapest_material)
        materials.remove(cheapest_material)
    return sorted_materials


def search_materials(state: State) -> None:
    """Find every material that satisfies the constraints and store the results.

    Loops over the catalog in ``state.materials``, keeps the materials that fail
    no constraint, sorts them from cheapest to most expensive, and saves them in
    ``state.matches``. A SearchRecord is also added to ``state.history``.

    Args:
        state: The current State. Its ``matches`` and ``history`` are modified.
    """
    found_materials = []
    for material in state.materials:
        if len(find_failed_constraints(material, state)) == 0:
            found_materials.append(material)

    found_materials = sort_materials_by_cost(found_materials, state.volume)
    state.matches = found_materials

    if len(found_materials) == 0:
        outcome = NO_MATCH_MESSAGE
    else:
        outcome = ", ".join([material.name for material in found_materials])
    state.history.append(SearchRecord(describe_constraints(state), outcome))


def describe_constraints(state: State) -> str:
    """Build a one-line summary of the user's current constraints.

    Args:
        state: The current State.

    Returns:
        Text such as "Weight <= 500.0 lbs, Cost <= $200.0, ...".
    """
    # round(number, 2) keeps at most 2 decimal places, and str() turns a
    # number into text so it can be joined to other text with +.
    weight_text = "Weight <= " + str(round(state.max_weight, 2)) + " lbs"
    cost_text = "Cost <= $" + str(round(state.max_cost, 2))
    volume_text = "Volume = " + str(round(state.volume, 2)) + " cu ft"
    stress_text = "Stress = " + str(state.required_strength) + " psi"
    location_text = "Location = " + state.site.name

    summary = (weight_text + ", " + cost_text + ", " + volume_text + ", "
               + stress_text + ", " + location_text)
    return summary


def text_for_default(number: float) -> str:
    """Turn a stored number into text for a TextBox, or blank if not set yet.

    Args:
        number: A stored constraint value (0 means "not entered yet").

    Returns:
        The number as text, or an empty string if the number is zero.
    """
    if number == 0:
        return ""
    return str(number)


def build_constraint_page(state: State, message: str) -> Page:
    """Build the page where the user enters all five constraints.
```

```
    Args:
        state: The current State (used to pre-fill the input fields).
        message: An error message to show at the top, or "" for none.

    Returns:
        The constraint input Page.
    """
    return Page(state, [
        Header("Enter Your Constraints"),
        message,
        "Maximum weight of the object (lbs):",
        TextBox("weight_text", text_for_default(state.max_weight)),
        "Maximum budget for material (dollars):",
        TextBox("cost_text", text_for_default(state.max_cost)),
        "Volume of material needed (cubic feet):",
        TextBox("volume_text", text_for_default(state.volume)),
        "Stress the object must withstand (psi):",
        TextBox("stress_text", text_for_default(state.required_strength)),
        "Project location:",
        SelectBox("location_name",
                  [site.name for site in SITE_OPTIONS],
                  state.site.name),
        LineBreak(),
        Button("Find Material", find_material),
        Button("Back to Home", index),
    ])


def build_results_page(state: State, message: str) -> Page:
    """Build the page that shows the constraints, the chosen material(s), and
    the controls for changing any single constraint.

    Args:
        state: The current State, including the latest ``matches``.
        message: An error message to show at the top, or "" for none.

    Returns:
        The results Page.
    """
    # Part 1: the user's constraints
    # Turn each value into text first, so each row is easy to read below.
    weight_text = str(round(state.max_weight, 2)) + " lbs"
    cost_text = "$" + str(round(state.max_cost, 2))
    volume_text = str(round(state.volume, 2)) + " cubic feet"
    stress_text = str(state.required_strength) + " psi"
    location_text = (state.site.name + " (" +
                     str(state.site.average_temperature) + " F, " +
                     state.site.weather + ")")

    # Each row of the table is a list with two items: a label and a value.
    # The first row is the table's column headings.
    constraint_rows = []
    constraint_rows.append(["Constraint", "Your Value"])
    constraint_rows.append(["Maximum weight", weight_text])
    constraint_rows.append(["Maximum cost", cost_text])
    constraint_rows.append(["Volume", volume_text])
    constraint_rows.append(["Stress", stress_text])
    constraint_rows.append(["Location", location_text])

    # Part 2: the outcome
    outcome_content = []
    if len(state.matches) == 0:
        outcome_content.append(Header(NO_MATCH_MESSAGE, 2))
        outcome_content.append("Try raising the budget or weight limit, "
                               "lowering the stress, or changing the location.")
    else:
        if len(state.matches) == 1:
            outcome_content.append(Header("Best Material", 2))
            outcome_content.append(state.matches[0].name + " is the only "
                                   "material that meets all your constraints.")
        else:
            outcome_content.append(Header("Suitable Materials", 2))
```

```
            outcome_content.append(str(len(state.matches)) +
                                   " materials meet all your constraints, so "
                                   "any of them could work. They are listed "
                                   "cheapest first.")

        match_rows = [["Material", "Weight (lbs)", "Total Cost",
                       "Strength (psi)"]]
        for material in state.matches:
            weight_text = str(round(calculate_weight(material, state.volume), 2))
            cost_text = "$" + str(round(calculate_cost(material, state.volume), 2))
            strength_text = str(material.strength)
            match_rows.append([material.name, weight_text, cost_text,
                               strength_text])
        outcome_content.append(Table(match_rows))

    # Part 3: controls for changing one constraint at a time
    return Page(state, [
        Header("Your Results"),
        message,
        Header("Your Constraints", 2),
        Table(constraint_rows),
        *outcome_content,
        HorizontalRule(),
        Header("Change One Constraint", 2),
        "New maximum weight (lbs):",
        TextBox("new_weight", ""),
        Button("Change Weight", change_weight),
        LineBreak(),
        "New maximum cost (dollars):",
        TextBox("new_cost", ""),
        Button("Change Cost", change_cost),
        LineBreak(),
        "New volume (cubic feet):",
        TextBox("new_volume", ""),
        Button("Change Volume", change_volume),
        LineBreak(),
        "New stress (psi):",
        TextBox("new_stress", ""),
        Button("Change Stress", change_stress),
        LineBreak(),
        "New location:",
        SelectBox("new_location",
                  [site.name for site in SITE_OPTIONS],
                  state.site.name),
        Button("Change Location", change_location),
        HorizontalRule(),
        Button("Start New Search", new_search),
        Button("Back to Home", index),
    ])


# ---------------------------------------------------------------------------
# Routes / pages
# ---------------------------------------------------------------------------

@route
def index(state: State) -> Page:
    """Home page: links to the instructions, a new search, and the lists.

    Args:
        state: The current State.

    Returns:
        The home Page.
    """
    return Page(state, [
        Header("Material Finder"),
        "Tell us about your object and where it will be used, and we will "
        "find the materials that fit.",
        LineBreak(),
        Button("Instructions", instructions_page),
        Button("Find a Material", constraint_page),
```

```
        Button("See All Materials", materials_page),
        Button("Past Searches", history_page),
    ])


@route
def instructions_page(state: State) -> Page:
    """Instructions page: explains each input and how the matching works.

    Args:
        state: The current State.

    Returns:
        The instructions Page.
    """
    return Page(state, [
        Header("Instructions"),
        "Fill in all five constraints. Numbers must be positive and contain "
        "only digits and at most one decimal point (no units or commas).",
        BulletedList([
            "Maximum weight: the heaviest the finished object can be, in pounds.",
            "Maximum cost: the most you can spend on the material, in dollars.",
            "Volume: how much material you need, in cubic feet.",
            "Stress: how much force per square inch the object must handle "
            "(psi). Whole numbers only.",
            "Location: pick where the object will be used. Wet locations need "
            "water resistant materials, and the material must be safe at "
            "the location's temperature.",
        ]),
        "A material is only suggested if it meets every constraint. If more "
        "than one material works, all of them are shown and any could work. "
        "If none work, you will be told no suitable material was found.",
        "On the results page you can change one constraint at a time to see "
        "how it affects the answer.",
        LineBreak(),
        Button("Back to Home", index),
    ])


@route
def constraint_page(state: State) -> Page:
    """Constraint input page: where the user enters the first set of constraints.

    Args:
        state: The current State.

    Returns:
        The constraint input Page.
    """
    return build_constraint_page(state, "")


@route
def find_material(state: State, weight_text: str, cost_text: str,
                  volume_text: str, stress_text: str,
                  location_name: str) -> Page:
    """Validate all five constraints, save them, and search for materials.

    Args:
        state: The current State. All five constraint fields are modified.
        weight_text: Text from the maximum weight TextBox.
        cost_text: Text from the maximum cost TextBox.
        volume_text: Text from the volume TextBox.
        stress_text: Text from the stress TextBox.
        location_name: Name chosen in the location SelectBox.

    Returns:
        The results Page if every input is valid, otherwise the constraint
        input Page with an error message.
    """
    if not is_positive_number(weight_text):
        return build_constraint_page(state, "Weight must be a positive number.")
```

```
    if not is_positive_number(cost_text):
        return build_constraint_page(state, "Cost must be a positive number.")
    if not is_positive_number(volume_text):
        return build_constraint_page(state, "Volume must be a positive number.")
    if not stress_text.strip().isdigit() or int(stress_text) <= 0:
        return build_constraint_page(state,
                                     "Stress must be a positive whole number.")

    state.max_weight = float(weight_text)
    state.max_cost = float(cost_text)
    state.volume = float(volume_text)
    state.required_strength = int(stress_text)
    state.site = find_site_by_name(location_name)

    search_materials(state)
    return build_results_page(state, "")


@route
def materials_page(state: State) -> Page:
    """List page: shows every material in the catalog.

    Args:
        state: The current State (its ``materials`` list is displayed).

    Returns:
        The catalog Page.
    """
    catalog_rows = [["Material", "Density (lb/cu ft)", "Cost ($/lb)",
                     "Water Resistant", "Strength (psi)", "Safe Temp (F)"]]
    for material in state.materials:
        if material.is_water_resistant:
            water_text = "Yes"
        else:
            water_text = "No"
        density_text = str(material.density)
        cost_text = "$" + str(material.cost_per_pound)
        strength_text = str(material.strength)
        temperature_text = (str(material.min_temperature) + " to " +
                            str(material.max_temperature))
        catalog_rows.append([material.name, density_text, cost_text,
                             water_text, strength_text, temperature_text])
    return Page(state, [
        Header("All Materials"),
        "The site can recommend any of these " + str(len(state.materials)) +
        " materials.",
        Table(catalog_rows),
        LineBreak(),
        Button("Back to Home", index),
    ])


@route
def history_page(state: State) -> Page:
    """History page: shows every search made during this visit.

    Args:
        state: The current State (its ``history`` list is displayed).

    Returns:
        The history Page.
    """
    if len(state.history) == 0:
        history_content = ["You have not made any searches yet."]
    else:
        history_rows = [["#", "Constraints", "Result"]]
        search_number = 1
        for record in state.history:
            history_rows.append([str(search_number),
                                 record.constraints_summary,
                                 record.outcome])
            search_number += 1
```

```
        history_content = [Table(history_rows)]
    return Page(state, [
        Header("Past Searches"),
        *history_content,
        LineBreak(),
        Button("Back to Home", index),
    ])


@route
def change_weight(state: State, new_weight: str) -> Page:
    """Change only the maximum weight and search again.

    Args:
        state: The current State. ``max_weight`` is modified.
        new_weight: Text from the new weight TextBox.

    Returns:
        The updated results Page, or the unchanged results Page with an
        error message if the input was invalid.
    """
    if not is_positive_number(new_weight):
        return build_results_page(state, "Weight must be a positive number.")
    state.max_weight = float(new_weight)
    search_materials(state)
    return build_results_page(state, "")


@route
def change_cost(state: State, new_cost: str) -> Page:
    """Change only the maximum cost and search again.

    Args:
        state: The current State. ``max_cost`` is modified.
        new_cost: Text from the new cost TextBox.

    Returns:
        The updated results Page, or the unchanged results Page with an
        error message if the input was invalid.
    """
    if not is_positive_number(new_cost):
        return build_results_page(state, "Cost must be a positive number.")
    state.max_cost = float(new_cost)
    search_materials(state)
    return build_results_page(state, "")


@route
def change_volume(state: State, new_volume: str) -> Page:
    """Change only the volume and search again.

    Args:
        state: The current State. ``volume`` is modified.
        new_volume: Text from the new volume TextBox.

    Returns:
        The updated results Page, or the unchanged results Page with an
        error message if the input was invalid.
    """
    if not is_positive_number(new_volume):
        return build_results_page(state, "Volume must be a positive number.")
    state.volume = float(new_volume)
    search_materials(state)
    return build_results_page(state, "")


@route
def change_stress(state: State, new_stress: str) -> Page:
    """Change only the required stress and search again.

    Args:
        state: The current State. ``required_strength`` is modified.
```

```
        new_stress: Text from the new stress TextBox.

    Returns:
        The updated results Page, or the unchanged results Page with an
        error message if the input was invalid.
    """
    if not new_stress.strip().isdigit() or int(new_stress) <= 0:
        return build_results_page(state,
                                  "Stress must be a positive whole number.")
    state.required_strength = int(new_stress)
    search_materials(state)
    return build_results_page(state, "")


@route
def change_location(state: State, new_location: str) -> Page:
    """Change only the project location and search again.

    Args:
        state: The current State. ``site`` is modified.
        new_location: Name chosen in the new location SelectBox.

    Returns:
        The updated results Page.
    """
    state.site = find_site_by_name(new_location)
    search_materials(state)
    return build_results_page(state, "")


@route
def new_search(state: State) -> Page:
    """Clear the old constraints and send the user to enter new ones.

    The search history and the materials catalog are kept.

    Args:
        state: The current State. All constraints and ``matches`` are reset.

    Returns:
        The constraint input Page with blank fields.
    """
    state.max_weight = 0.0
    state.max_cost = 0.0
    state.volume = 0.0
    state.required_strength = 0
    state.matches = []
    return build_constraint_page(state, "")


# ---------------------------------------------------------------------------
# Start the website
# ---------------------------------------------------------------------------

start_server(State(
    materials=build_material_catalog(),
    max_weight=0.0,
    max_cost=0.0,
    volume=0.0,
    required_strength=0,
    site=SITE_OPTIONS[0],
    matches=[],
    history=[],
))
```